Ingegneria del Software - Politecnico di Milano

MESOS

Documentazione dell'Architettura di Rete

Protocollo di Comunicazione Client-Server via TCP

Contents

1 Panoramica dell'Architettura **3**
- 1.1 Modello Architetturale . . . 3
- 1.2 Visione d'Insieme — MVC Distribuito . . . 4
- 1.3 Pacchetti Principali . . . 4

2 Componenti di Rete **5**
- 2.1 Lato Server . . . 5
  - 2.1.1 ConnectionAcceptor . . . 5
  - 2.1.2 VirtualViewTCP . . . 5
  - 2.1.3 ClientCommandHandler . . . 5
  - 2.1.4 ConnectionContext . . . 6
- 2.2 Lato Client . . . 6
  - 2.2.1 VirtualServerTCP . . . 6
  - 2.2.2 TcpServerMessageReceiver . . . 6
  - 2.2.3 ServerMessageListener . . . 6

3 Formato dei Messaggi **7**
- 3.1 Struttura Generale . . . 7
- 3.2 ClientMessage — Dal Client al Server . . . 7
  - 3.2.1 Struttura . . . 7
  - 3.2.2 Tipi disponibili (ClientMessageType) . . . 8
- 3.3 ServerMessage — Dal Server al Client . . . 8
  - 3.3.1 Struttura . . . 8
  - 3.3.2 Tipi disponibili (ServerMessageType) . . . 9
- 3.4 Payload Dettagliati . . . 9
  - 3.4.1 AuthenticationPayload (Client → Server) . . . 9
  - 3.4.2 PreMatchUpdatePayload (Server → Client) . . . 10
  - 3.4.3 GamePayload (Server → Client) . . . 10
  - 3.4.4 ErrorPayload (Server → Client) . . . 10

4 Flusso di Comunicazione **11**
- 4.1 Diagramma di Sequenza Completo . . . 11
- 4.2 Fasi della Comunicazione . . . 12
  - 4.2.1 Fase 1 — Connessione TCP . . . 12
  - 4.2.2 Fase 2 — Autenticazione . . . 12
  - 4.2.3 Fase 3 — Loop di Gioco . . . 12
  - 4.2.4 Fase 4 — Notifiche Observer . . . 12

4.3 Gestione degli Errori 12

5 Pattern Architetturali **13**
5.1 Proxy Pattern — VirtualView 13
5.2 Observer Pattern — Notifiche di Stato 13
5.3 Strategy Pattern — TCP vs RMI 13

6 Considerazioni Tecniche **14**
6.1 Threading Model 14
6.2 Serializzazione JSON con Gson 14
6.3 Sicurezza e Raccomandazioni 15
6.4 Estensibilità del Protocollo 15

Chapter 1

Panoramica dell'Architettura

1.1 Modello Architetturale

Il progetto adotta un'architettura **Client-Server distribuita** ispirata al pattern **MVC (Model-View-Controller)**, in cui le responsabilità sono divise tra macchina client e macchina server. La comunicazione avviene interamente tramite **socket TCP**, garantendo affidabilità, ordine di consegna e connessioni persistenti e bidirezionali.

Principi Chiave dell'Architettura

- **Trasporto:** TCP — connessioni persistenti e bidirezionali
- **Serializzazione:** JSON via libreria Gson
- **Modello:** Client-Server con MVC distribuito lato server
- **Concorrenza:** un thread per connessione client (server-side) + thread listener lato client
- **Estensibilità:** supporto futuro a RMI tramite pattern Proxy/Strategy

1.2 Visione d'Insieme — MVC Distribuito

Il cuore del design è un *MVC distribuito server-side*, in cui la **VirtualView** funge da proxy trasparente per ogni client connesso. Il server interagisce con le VirtualView come se fossero client locali; è la VirtualView stessa a occuparsi di serializzare e trasmettere i dati sulla rete.

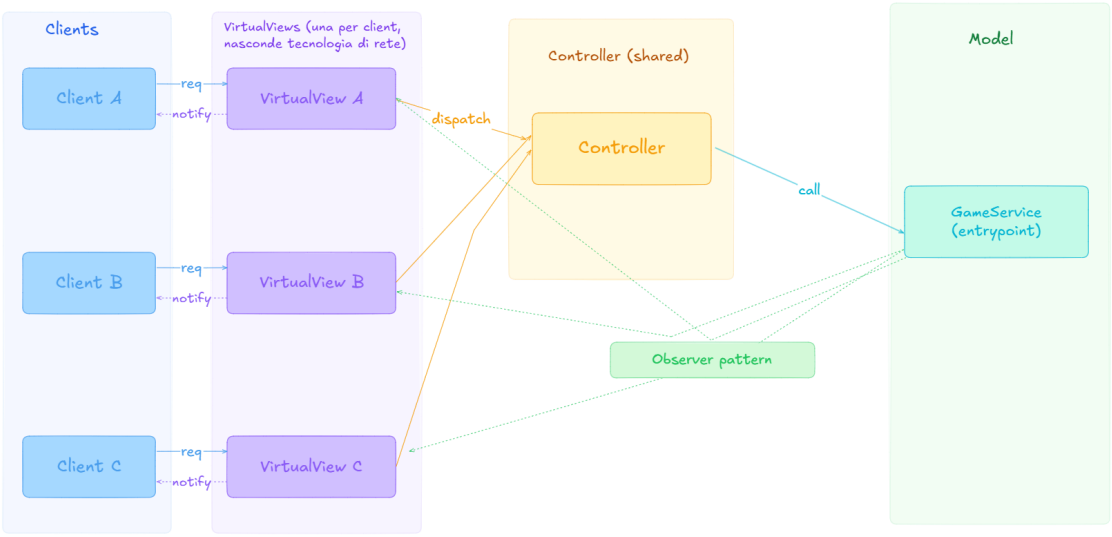


Figure 1.1: Visione d'insieme dell'architettura MVC distribuita

1.3 Pacchetti Principali

Pacchetto	Responsabilità
`server.network`	Accettazione connessioni TCP, threading per client
`server.virtualview`	VirtualView per ogni client (proxy + observer)
`client.virtualserver`	Connessione al server, invio e ricezione messaggi
`shared.messages`	Definizione di `ClientMessage` e `ServerMessage`
`shared.payload`	Payload tipizzati (client-side e server-side)

Table 1.1: Struttura dei pacchetti principali

Chapter 2

Componenti di Rete

2.1 Lato Server

2.1.1 ConnectionAcceptor

Il **ConnectionAcceptor** è il punto di ingresso unico per le connessioni di rete. Rimane in ascolto su una porta TCP configurabile (default: `8080`) in un ciclo continuo (`while(true)`). Ad ogni nuova connessione accettata:

1. Ottiene il `Socket` del client connesso
2. Instanzia un nuovo oggetto `VirtualViewTCP`
3. Avvia un thread dedicato per quella connessione

Identificazione connessione: ogni socket è univocamente identificato dalla quadrupla `(IP client, Porta client, IP server, Porta server)`.

2.1.2 VirtualViewTCP

La **VirtualViewTCP** gestisce l'intera vita della comunicazione con un singolo client. Implementa due pattern fondamentali:

- **Proxy Pattern:** rappresenta il client agli occhi di Controller e Model — questi pensano di comunicare direttamente con il client, ma in realtà passano attraverso la VirtualView che nasconde la rete.
- **Observer Pattern:** è registrata come osservatore sul Model; quando lo stato cambia, il Model notifica la VirtualView, che provvede a serializzare e trasmettere l'aggiornamento al client.

2.1.3 ClientCommandHandler

Il **ClientCommandHandler** riceve la stringa JSON dal socket, la parsa con Gson, determina il `ClientMessageType`, esegue la logica di business chiamando il ServerController, e costruisce il `ServerMessage` di risposta.

2.1.4 ConnectionContext

Il **ConnectionContext** mantiene lo stato della sessione: utente autenticato, fase corrente del gioco, lobby associata. Viene passato all'handler per decisioni contestuali (es. un messaggio MOVE_TOTEM è valido solo se la partita è in corso).

2.2 Lato Client

2.2.1 VirtualServerTCP

Il **VirtualServerTCP** è il punto di contatto del client con il server. Espone metodi ad alto livello (`authenticate()`, `createLobby()`, ecc.) che internamente costruiscono i `ClientMessage`, li serializzano in JSON e li scrivono sul socket.

2.2.2 TcpServerMessageReceiver

Il **TcpServerMessageReceiver** gira in un thread separato, in ascolto continuo sul socket. Quando arriva una riga JSON:

1. Legge la riga dal `BufferedReader`
2. Parsa il JSON in un `ServerMessage`
3. Invoca il callback `listener.onServerMessage(message)`

Questo design evita che la ricezione blocchi il thread principale del client.

2.2.3 ServerMessageListener

Interfaccia semplice con un unico metodo:

```
void onServerMessage(ServerMessage message);
```

La View del client implementa questa interfaccia per reagire agli aggiornamenti ricevuti.

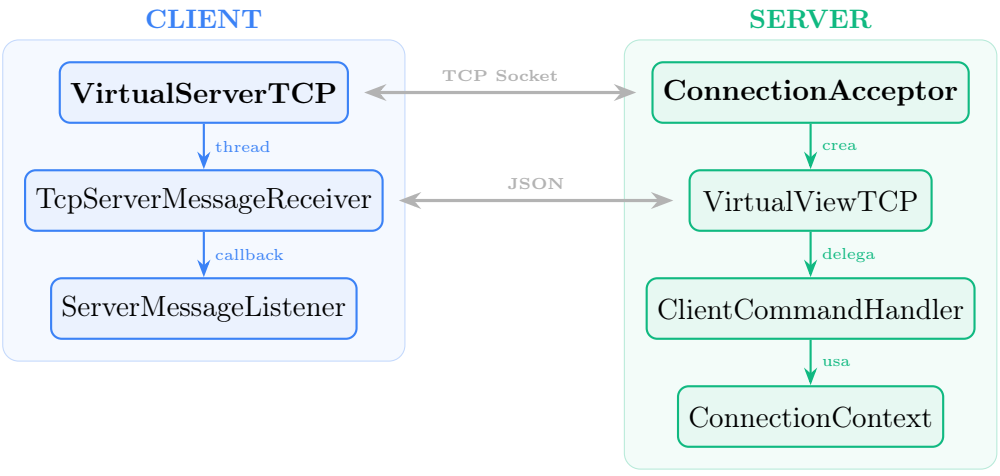


Figure 2.1: Componenti di rete client-server e loro relazioni

Chapter 3

Formato dei Messaggi

3.1 Struttura Generale

Tutti i messaggi scambiati tra client e server seguono un formato JSON uniforme con due campi obbligatori:

```
{
  "type": "NOME_TIPO",
  "payload": { ... }
}
```

Il campo `type` è un valore dell'enum corrispondente (`ClientMessageType` o `ServerMessageType`). Il campo `payload` è un oggetto specifico per quel tipo di messaggio, oppure `null`.

3.2 ClientMessage — Dal Client al Server

3.2.1 Struttura

Listing 3.1: Struttura di un ClientMessage

```
{
  "type": "AUTHENTICATE",
  "payload": {
    "nickname": "Player1"
  }
}
```

3.2.2 Tipi disponibili (ClientMessageType)

Tipo	Fase	Descrizione
AUTHENTICATE	Pre-partita	Autenticazione con nickname utente
CREATE_LOBBY	Pre-partita	Crea una nuova lobby di gioco
JOIN_LOBBY	Pre-partita	Si unisce a una lobby esistente
EXIT_LOBBY	Pre-partita	Abbandona la lobby corrente
START_LOBBY	Pre-partita	Avvia la partita nella lobby
SETUP_MATCH	Setup	Configurazione iniziale della partita
MOVE_TOTEM	In-game	Movimento del totem sulla board
CHOOSE_CARD	In-game	Selezione di una carta
EXTRA_DRAW	In-game	Pescata aggiuntiva
RESOLVE_EFFECTS	In-game	Risoluzione degli effetti pendenti
RESET_BOARD	In-game	Reset del tabellone
FINAL_SCORE	Fine	Richiesta punteggio finale

Table 3.1: Tipi di messaggi del client

3.3 ServerMessage — Dal Server al Client

3.3.1 Struttura

Listing 3.2: ServerMessage: aggiornamento pre-partita

```
{
  "type": "PRE_MATCH_UPDATE",
  "payload": {
    "event": "AUTHENTICATED",
    "nickname": "Player1",
    "message": "Authentication completed successfully",
    "lobbyId": null,
    "players": [],
    "host": null,
    "maxPlayers": 0
  }
}
```

Listing 3.3: ServerMessage: errore

```
{
  "type": "ERROR",
  "payload": {
    "code": "AUTH_FAILED",
    "message": "Nickname gia' in uso"
  }
}
```

Listing 3.4: ServerMessage: aggiornamento di gioco

```
{
  "type": "GAME_UPDATE",
  "payload": {
    "id": "match-42",
    "status": "IN_PROGRESS",
    "currentEra": 2,
    "turnOrder": ["Player1", "Player2"],
    "turnState": "MOVE_TOTEM",
    "players": { ... },
    "board": { ... }
  }
}
```

3.3.2 Tipi disponibili (ServerMessageType)

Tipo	Descrizione
ERROR	Errore con codice e messaggio descrittivo
PRE_MATCH_UPDATE	Aggiornamento di stato pre-partita (lobby, autenticazione)
GAME_UPDATE	Snapshot completo dello stato di gioco aggiornato

Table 3.2: Tipi di messaggi del server

3.4 Payload Dettagliati

3.4.1 AuthenticationPayload (Client → Server)

Campo	Tipo	Descrizione
nickname	String	Nome utente univoco del giocatore

3.4.2 PreMatchUpdatePayload (Server → Client)

Campo	Tipo	Descrizione
`event`	String	Tipo di evento (es. AUTHENTICATED, LOBBY_CREATED)
`nickname`	String	Nickname del giocatore coinvolto
`message`	String	Messaggio descrittivo leggibile
`lobbyId`	String?	ID della lobby (se applicabile)
`players`	List	Lista giocatori nella lobby
`host`	String?	Nickname dell'host della lobby
`maxPlayers`	int	Numero massimo di giocatori

3.4.3 GamePayload (Server → Client)

Campo	Tipo	Descrizione
`id`	String	Identificatore univoco della partita
`status`	Enum	Stato corrente (SETUP, IN_PROGRESS, ENDED)
`currentEra`	int	Era corrente del gioco
`turnOrder`	List	Ordine dei turni dei giocatori
`turnState`	Enum	Stato del turno (es. MOVE_TOTEM, CHOOSE_CARD)
`players`	Map	Stato di ogni giocatore (score, mano, ecc.)
`board`	Object	Stato completo del tabellone

3.4.4 ErrorPayload (Server → Client)

Campo	Tipo	Descrizione
`code`	String	Codice errore machine-readable (es. AUTH_FAILED)
`message`	String	Descrizione leggibile dell'errore

Chapter 4

Flusso di Comunicazione

4.1 Diagramma di Sequenza Completo

Il seguente diagramma illustra l'intero ciclo di vita di una comunicazione, dalla connessione TCP all'elaborazione della risposta.

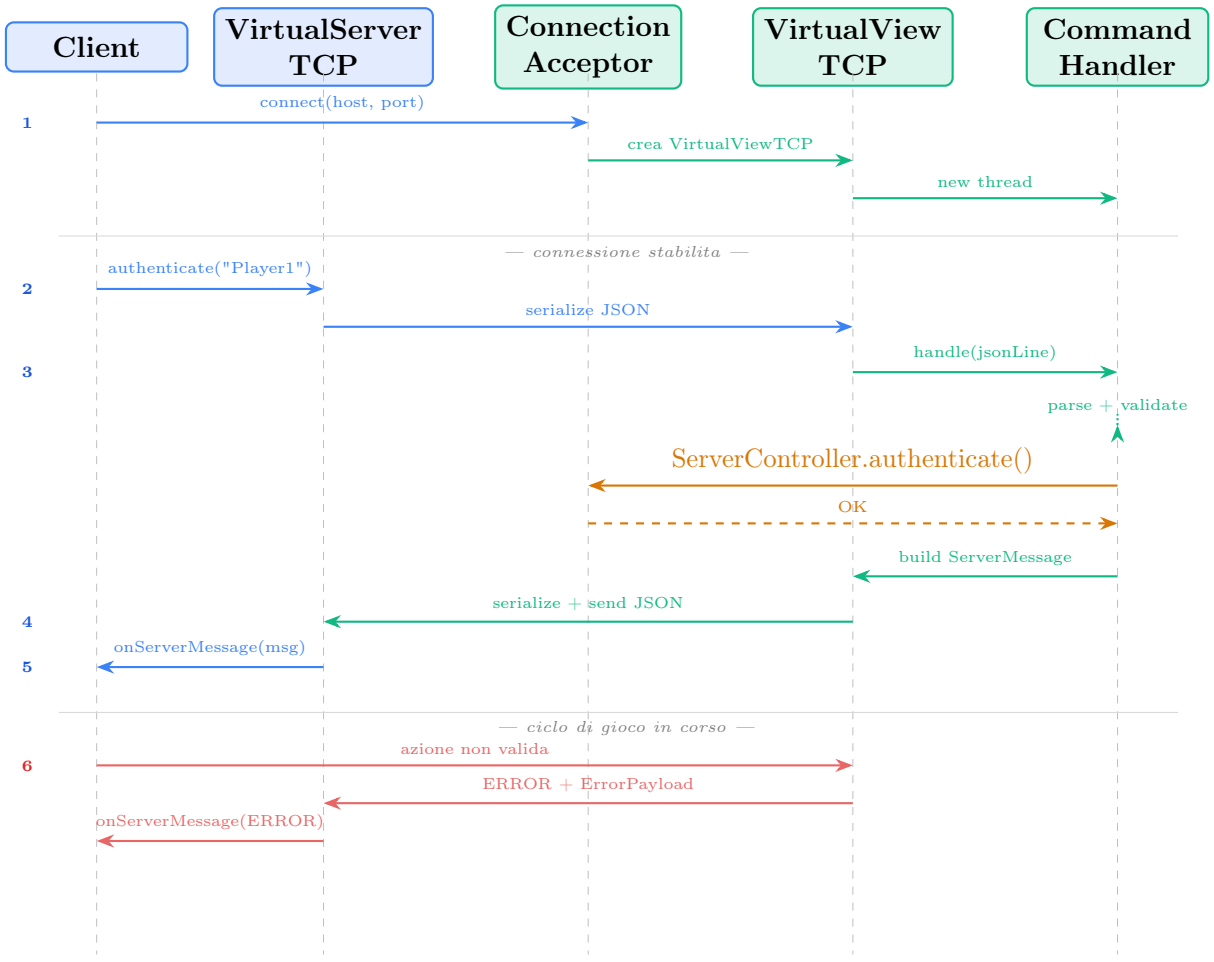


Figure 4.1: Diagramma di sequenza della comunicazione client-server

4.2 Fasi della Comunicazione

4.2.1 Fase 1 — Connessione TCP

1. Il **client** invoca `new VirtualServerTCP(host, port, listener)`.
2. Il socket TCP viene aperto verso l'IP/porta del server.
3. Il **ConnectionAcceptor** accetta la connessione e istanzia una `VirtualViewTCP`.
4. Viene creato un thread dedicato per gestire quella connessione.

4.2.2 Fase 2 — Autenticazione

Il primo messaggio che ogni client deve inviare è `AUTHENTICATE`. Fino al completamento dell'autenticazione, qualunque altro messaggio viene rifiutato con un `ERROR`.

4.2.3 Fase 3 — Loop di Gioco

Una volta autenticato, il client partecipa al flusso di gioco: può creare/joinare lobby, configurare la partita e inviare mosse. Il server risponde con `GAME_UPDATE` dopo ogni azione che modifica lo stato, e con `ERROR` in caso di azione non valida.

4.2.4 Fase 4 — Notifiche Observer

Quando il Model subisce modifiche (es. un altro giocatore ha fatto una mossa), le VirtualView di tutti i client osservatori ricevono la notifica e inviano automaticamente un `GAME_UPDATE` a ciascun client.

4.3 Gestione degli Errori

Principio: le eccezioni non atterrano mai sul client come stack trace. Ogni eccezione lato server viene intercettata e trasformata in un `ServerMessage` di tipo `ERROR` con `ErrorPayload` strutturato.

Scenari di errore gestiti:

- Nickname già in uso → `AUTH_FAILED`
- Messaggio con tipo sconosciuto → `UNKNOWN_MESSAGE_TYPE`
- Azione non valida per la fase corrente → `INVALID_ACTION`
- Payload malformato (JSON non valido) → `PARSE_ERROR`
- Disconnessione imprevista del client → cleanup lato server
- Altri errori saranno gestiti in maniera più generale in quanto sarebbe dispendioso andare a separare ogni singola azione

Chapter 5

Pattern Architetturali

5.1 Proxy Pattern — VirtualView

La VirtualView implementa il **Proxy Pattern**: controller e model non sanno se stanno comunicando con un client locale o remoto. La VirtualView è l'unico punto in cui la rete esiste.

5.2 Observer Pattern — Notifiche di Stato

Il Model implementa il **Observer Pattern**: ogni VirtualView è registrata come observer. Quando lo stato del gioco cambia, il Model notifica tutte le VirtualView registrate, che propagano l'aggiornamento ai rispettivi client.

5.3 Strategy Pattern — TCP vs RMI

L'architettura è progettata per supportare un cambio a runtime del protocollo di trasporto. Sia `VirtualViewTCP` (server-side) che `VirtualServerTCP` (client-side) implementano interfacce comuni, rendendo possibile sostituire TCP con RMI senza modificare la logica di business.

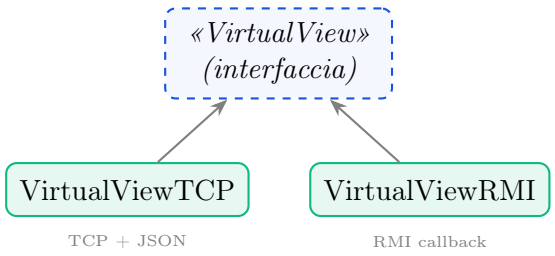


Figure 5.1: Strategy Pattern — intercambiabilità TCP/RMI

Chapter 6

Considerazioni Tecniche

6.1 Threading Model

Componente	Threading
ConnectionAcceptor	Thread principale del server, bloccante su `accept()`
VirtualViewTCP	Un thread per ogni client connesso
ClientCommandHandler	Gira nel thread della VirtualViewTCP
TcpServerMessageReceiver	Thread separato nel client

Table 6.1: Modello di threading

Nota sulla scalabilità: il modello "un thread per client" è semplice e adeguato per un numero limitato di giocatori. Per carichi elevati, si consiglia di adottare un **thread pool** (es. `ExecutorService`) o un approccio NIO non bloccante.

6.2 Serializzazione JSON con Gson

La libreria **Gson** di Google converte automaticamente oggetti Java in stringhe JSON e viceversa. I messaggi vengono trasmessi sul socket come **singole righe** (terminazione `\n`), per permettere a `BufferedReader.readLine()` di delimitarli correttamente.

6.3 Sicurezza e Raccomandazioni

Il protocollo attuale usa **TCP plain** senza crittografia. Per un deployment in produzione è fortemente consigliato:

- Aggiungere **TLS/SSL** tramite `SSLServerSocket` / `SSLSocket`
- Implementare autenticazione con token (es. JWT) anziché solo nickname
- Validare la dimensione massima dei payload per prevenire attacchi DoS

6.4 Estensibilità del Protocollo

Il protocollo è progettato per essere esteso agevolmente:

1. **Nuovo tipo di messaggio:** aggiungere un valore all'enum `ClientMessageType` o `ServerMessageType`.
2. **Nuovo payload:** creare una classe POJO nel pacchetto `shared.payload` (Gson la serializza automaticamente).
3. **Nuova fase di gioco:** aggiungere la logica nel `ClientCommandHandler` e aggiornare il `ConnectionContext`.